

Reviewer Pack — Minimal p-Adic Root Count and Communication Complexity Rank ...

Entry 5812d7d4f04d · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-06 10:58:00 UTC

Minimal p-Adic Root Count and Communication Complexity Rank Variance

Entry ID: 5812d7d4f04d **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-06 10:58:00 UTC

1. Conjecture

Field A (mathematical branch): p-adic Analysis **Field B** (complexity object): Communication Complexity

Statement:

For every n-bit communication protocol φ , the number of distinct p-adic roots modulo p^2 that appear in the characteristic polynomial of the communication matrix of φ is upper-bounded by a constant factor times the variance of the rank of the communication matrix, i.e., $|N_{\text{p-adic}}(\varphi)| \leq c \cdot \text{Var}(\text{Rank}(\varphi))$

Rationale (proposer's reasoning):

p-adic analysis provides a framework for studying number-theoretic properties that can be applied to the study of discrete structures in complexity theory. The p-adic root count could serve as an invariant related to the algebraic structure of communication matrices, which may reveal new insights into the rank variance and thus the complexity of the protocol.

Taxonomy category: p-adic_analysis_communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ab459d67c9e07f3c

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For each n-bit communication protocol φ , if $|N_{\text{p-adic}}(\varphi)| / \text{Var}(\text{Rank}(\varphi)) \leq c$ for all seeds and $c \leq 3$, then the conjecture is supported.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.95	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(matrix):
    rows, cols = len(matrix), len(matrix[0])
    for i in range(rows):
        # Find pivot
        max_row = i
        for r in range(i+1, rows):
            if abs(matrix[r][i]) > abs(matrix[max_row][i]):
                max_row = r
        matrix[i], matrix[max_row] = matrix[max_row], matrix[i]

        # Eliminate below
        for r in range(i+1, rows):
            factor = -matrix[r][i] / matrix[i][i]
            for c in range(cols):
                if i == c:
                    matrix[r][c] = 0
                else:
                    matrix[r][c] += factor * matrix[i][c]

def determinant(matrix):
    n = len(matrix)
    det = Fraction(1)
    for i in range(n):
        det *= matrix[i][i]
    return det

def characteristic_polynomial(matrix):
    n = len(matrix)
    x = Fraction('x')
    identity = [[Fraction(0) if r != c else Fraction(1) for c in range(n)] for r in range(n)]

    # Augmented matrix
    aug_matrix = [row + identity[i] for i, row in enumerate(matrix)]
```

```

# Perform Gaussian elimination
gaussian_elimination(aug_matrix)

# Calculate determinant of the upper triangular matrix
char_poly = determinant([[aug_matrix[r][c] for c in range(n)] for r in range(n)])

return char_poly

def rank(matrix):
    n, m = len(matrix), len(matrix[0])
    row_rref = [row[:] for row in matrix]
    rank = 0

    for i in range(m):
        if sum(row[i] != Fraction(0) for row in row_rref[:n]):
            rank += 1
            # Make the leading coefficient 1
            lead_coeff = row_rref[rank-1][i]
            for j in range(n):
                row_rref[j][i] /= lead_coeff

            # Eliminate above and below
            for r in range(rank-1):
                factor = -row_rref[r][i]
                for c in range(m):
                    if i == c:
                        row_rref[r][c] = 0
                    else:
                        row_rref[r][c] += factor * row_rref[rank-1][c]

    return rank

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([5, 10, 15, 20, 30, 40])
    communication_matrix = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]

    char_poly = characteristic_polynomial(communication_matrix)
    rank_val = rank(communication_matrix)

    # Count distinct p-adic roots modulo p^2
    p = 2
    p_adic_roots = set()
    for coeff in char_poly.numerator.coefficients:
        if coeff % p == 0:
            root = -coeff / (p**2)
            if root not in p_adic_roots:
                p_adic_roots.add(root)

    n_max = n
    instances_tested = 1

    # Calculate the ratio  $|N_{\{p\text{-adic}\}}(\varphi)| / \text{Var}(\text{Rank}(\varphi))$ 
    mean_ratio = len(p_adic_roots) / (rank_val * (n - rank_val))

```

```

conjecture_holds = mean_ratio <= 3
counterexample = "" if conjecture_holds else f"mean_ratio={mean_ratio}"

return {
    "metric_name": "mean_ratio",
    "metric_value": mean_ratio,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = math.sqrt(sum((result["metric_value"] - mean_metric_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results) and support_fraction >= 0.8:
        first_failing_seed = next(seed for seed, result in enumerate(results, start=1) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mean_ratio' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_evidence")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a877e0fa.py", line 125, in <module>

```

    trial_result = run_trial(seed)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a877e0fa.py", line 89, in run_trial

```

    char_poly = characteristic_polynomial(communication_matrix)
    ~~~~~
```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a877e0fa.py", line 46, in characteristic_polynomial

```

    x = Fraction('x')
    ~~~~~
```

File "/usr/lib/python3.12/fractions.py", line 239, in __new__

```

    raise ValueError('Invalid literal for Fraction: %r' % s)
```

ValueError: Invalid literal for Fraction: 'x'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution before producing data to evaluate the conjecture.
| next: Investigate and fix the crash in the test code to proceed with the evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14043
2	propose	ollama_remote	glm4:latest	0	0	13237
3	preregistration	ollama_remote	glm4:latest	0	0	10935
4	novelty	ollama_remote	glm4:latest	0	0	8422
5	novelty	ollama_remote	glm4:latest	0	0	8358
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24303
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11831
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16305
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13855
10	judge	ollama_remote	glm4:latest	0	0	11501

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132790 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/5812d7d4f04d.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/5812d7d4f04d.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/5812d7d4f04d.tar.gz (if generated)